
TECHNICAL DOCUMENTATION: CINEMATCH

Comprehensive Guide for Developers and Project Reviewers

PROJECT OVERVIEW

CineMatch is an interactive web application designed to help users quickly decide what movie to watch. Many users experience “decision fatigue” when browsing streaming platforms due to overwhelming number of options available. CineMatch addresses this problem by providing personalized movie recommendations based on a user’s mood, available time, and optional release year preference.

The application is intended for casual movie viewers who want a faster and more guided way to choose content. Instead of endlessly scrolling, users can answer a few simple inputs and receive curated recommendations. CineMatch also enhances the experience by allowing users to view movie details, watch trailers, and track movies they have already watched to avoid repeating suggestions.

TECHNICAL ARCHITECTURE

CLASS DIAGRAM

CineMatch is structured using object-oriented programming principles with three main classes:

Movie Class

The **Movie** class represents individual movie objects retrieved from the TMDB API. It encapsulates movie-related data such as title, release date, rating, overview, and poster path. It also includes helper methods like retrieving the movie year and generating a full poster image URL.

RecommendationEngine Class

The **RecommendationEngine** class is responsible for managing all recommendation logic. It communicates with the TMDB API to fetch movie data based on user input. It also processes this data by filtering results based on time preference and limiting the number of displayed results. This class acts as the core logic layer of the application.

WatchlistManager Class

The **WatchlistManager** class manages the state of watched movies. It uses the browser’s localStorage to persist data so that users can refresh the page without losing their watched list. It provides methods to add, remove, and check for movies in the watchlist.

Relationships

- The **RecommendationEngine** creates and returns **Movie** objects.
- The **WatchlistManager** stores and retrieves **Movie** objects.
- The main application (app.js) connects all classes and manages user interaction.
- This structure separates concerns effectively, making the application easier to maintain and extend.

FEATURE WALKTHROUGH

User Quiz-Based Recommendations

Users can select mood, time availability, and optionally a release year. Based on these inputs, the application fetches, and filters movie data to generate personalized recommendations.

Search Functionality

Users can also manually search for movies by title. This provides flexibility for users who already have something in mind.

Dynamic Movie Results

Movie results are displayed as interactive cards showing posters, titles, ratings, and descriptions. These are generated dynamically using DOM manipulation.

Movie Details Modal

Clicking a movie card opens a modal that displays additional details and a trailer (if available). This allows users to explore content without leaving the page.

Trailer Integration

Trailers are embedded using YouTube if available from the TMDB API. This enhances the multimedia experience and helps users make informed decisions.

Watched List

Users can mark movies as they watched. These movies are saved using localStorage and displayed in a separate section. Watched movies are also removed from future recommendations.

Animations

GSAP is used to animate movie cards when results load and to animate the modal when it appears. These animations guide user attention and improve the overall user experience.

API DOCUMENTATION

CineMatch uses the [TMDB \(The Movie Database\) API](#) to retrieve movie data.

Endpoints Used

- | | |
|--|--|
| <ul style="list-style-type: none">• Search
/search/movie
Used to retrieve movie results based on mood keywords or search queries.• Movie
/movie/{id}/videos
Used to retrieve trailer data for a selected movie. | <div>Movies</div>
<div>Videos</div> |
|--|--|

Data Retrieved

The application uses:

- title
- poster_path
- release_date
- vote_average
- overview
- trailer video keys.

This data is used to populate movie cards and modal views.

Authentication

The API requires an API key, which is stored in the application as a constant. For security, the key is not shared publicly in documentation or repositories.

CHALLENGES AND DECISIONS

One of the most challenging aspects of the project was implementing the movie details modal and trailer functionality. Not all movies in the API have available trailers, which require missing data gracefully to prevent the application from breaking.

Another challenge was ensuring that the modal behaved correctly, including fixing issues where it would appear on page load or not close properly. Debugging these issues required careful attention to event handling and DOM structure.

A key design decision was to filter out watched movies from recommendations. This improves user experience by preventing duplicate suggestions and making the application feel more intelligent.

If more time were available, improvements could include:

- more advanced recommendation logic based on genres or user history.
- improved UI design and animations
- additional filters such as streaming platform availability
- user accounts instead of localStorage

HOW TO RUN THE APPLICATION LOCALLY

1. Download or clone the project files.
2. Open the project folder in a code editor such as Visual Studio Code.
3. Open the app.js file and insert a valid TMDB API key in the API_KEY variable.
4. Install and run a local server (such as the Live Server extension in VS Code).
5. Open index.html in the browser.
6. Use the quiz or search feature to interact with the application.